

EXP03

CODE:

```
import torch, torch.nn as nn, torch.optim as optim
import torchvision, torchvision.transforms as T
import matplotlib.pyplot as plt

# 1. Setup & Data
device = "cuda" if torch.cuda.is_available() else "cpu"
transform = T.Compose([T.ToTensor(), T.Normalize((0.5,), (0.5,))])
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
loader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)

# 2. Model & Training Function
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Flatten(), nn.Linear(3*32*32, 256), nn.ReLU(), nn.Linear(256, 10))
    def forward(self, x): return self.net(x)

def train(optimizer, epochs=10):
    model, criterion = MLP().to(device), nn.CrossEntropyLoss()
    losses, accs = [], []
    for epoch in range(epochs):
        total_loss, correct, total = 0, 0, 0
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            correct += (outputs.argmax(1) == labels.sum().item())
            total += labels.size(0)
        losses.append(total_loss / len(loader))
        accs.append(100 * correct / total)
    return losses, accs

# 3. Train Both Optimizers
sgd_opt = optim.SGD(MLP().parameters(), lr=0.01, momentum=0.9)
adam_opt = optim.Adam(MLP().parameters(), lr=0.001)

loss_sgd, acc_sgd = train(sgd_opt)
loss_adam, acc_adam = train(adam_opt)

# 4. Plot Results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.plot(loss_sgd, label="SGD + Momentum"); ax1.plot(loss_adam, label="Adam")
ax1.set_title("Loss Comparison"); ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss"); ax1.legend()

ax2.plot(acc_sgd, label="SGD + Momentum"); ax2.plot(acc_adam, label="Adam")
ax2.set_title("Accuracy Comparison (%)"); ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy");
ax2.legend()
```

```
plt.tight_layout(); plt.show()
```

OTUPUT:

